

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Proyecto Fase 2

Índice

1. Resumen	3
2. Objetivos	3
2.1 Objetivo General	3
2.2 Objetivos Específicos	3
3. Enunciado	4
3.1 Alcance específico de la Fase 2	4
3.2 Kubernetes: despliegue y operación	4
3.2.1 Manifiestos obligatorios	4
3.2.2 Probes, recursos y rollout/rollback	5
3.3 Seguridad con JWT	6
3.3.1 Servicio de autenticación	6
3.3.2 Autorización en microservicios y frontend	6
3.4 CI/CD: de la imagen al despliegue	7
3.4.1 Pipeline mínimo obligatorio	7
3.4.2 Publicación en registry	7
3.4.3 Despliegue automatizado a Kubernetes	8
3.5 Contratos y validación en CI	8
3.5.1 OpenAPI / JSON Schema	8
3.5.2 Validaciones automáticas en pipeline	8
3.6 Documentación de arquitectura y operación	9
4. Entregables	9
5. Restricciones técnicas	10
6. Documentación	10
7. Consideraciones adicionales	10
7.1 Historial de desarrollo	10
7.2 Cálculo de nota individual	11
7.3 Integridad académica	11
8. Fecha y Modo de entrega	11

1. Resumen

En esta fase el objetivo es **llevar EconoMarket a un nivel operativo completo sobre Kubernetes**, con un **ciclo de entrega automatizado** y una **seguridad basada en JWT claramente demostrable** desde el frontend hasta los microservicios.

No se redefinen el dominio ni los microservicios ni las tecnologías base: se parte de que ya existen y funcionan. El trabajo ahora consiste en:

- **Desplegar los servicios clave en un clúster Kubernetes**, usando manifiestos versionados dentro del repositorio.
- **Configurar correctamente pruebas, recursos y despliegues** para que el sistema sea observable y manejable (saber cuándo está listo, cuándo está sano y poder actualizarlo sin romperlo).
- **Aplicar JWT de forma consistente** en la ruta completa: autenticación, emisión de tokens, verificación en los servicios, protección de endpoints y uso real desde el frontend.
- **Extender el pipeline a CI/CD**, de forma que no solo construya y pruebe, sino que también publique imágenes en un registro y ejecute el despliegue a Kubernetes.
- **Fortalecer los contratos (OpenAPI/JSON Schema) y validarlos automáticamente en CI**, para reducir rupturas entre servicios y frente al frontend.
- **Documentar de forma precisa la arquitectura y la operación**, de modo que cualquier auxiliar pueda entender cómo está organizado el sistema y cómo se despliega y opera en el clúster.

2. Objetivos

2.1 Objetivo General

Operar EconoMarket en **Kubernetes** con un **pipeline CI/CD funcional** y con **seguridad JWT aplicada de extremo a extremo**, manteniendo la arquitectura por microservicios y garantizando que el sistema es desplegable, verificable y mantenible.

2.2 Objetivos Específicos

En esta fase se busca que cada equipo sea capaz de:

1. **Declarar el despliegue del sistema en Kubernetes** mediante manifiestos (YAML) claros, organizados y versionados.
2. **Configurar Kubernetes para operación real**, incluyendo readiness/liveness probes, requests/limits de recursos y mecanismos de rollout/rollback.
3. **Aplicar JWT como mecanismo de autenticación y autorización** en los microservicios y en el frontend, con un flujo completo de login → token → acceso a recursos protegidos.

4. **Construir un pipeline CI/CD** que, a partir de un cambio en el código, ejecute pruebas, construya imágenes, publique en un registro y despliegue al clúster de forma reproducible.
5. **Gestionar contratos Open API/JSON Schema** de forma que puedan validarse automáticamente en CI y sirvan como fuente de verdad para el frontend y otros consumidores.
6. **Generar documentación de arquitectura y operación** que explique decisiones, vista de despliegue, dependencias, configuración y pasos concretos de ejecución.

3. Enunciado

3.1 Alcance específico de la Fase 2

En esta fase el equipo debe entregar un sistema que:

1. **Siga funcionando funcionalmente como hasta ahora**, pero con foco en:
 - cómo se despliega,
 - cómo se protege con JWT,
 - cómo se entrega mediante CI/CD,
 - cómo se documenta su arquitectura y operación.
2. **Tenga un subconjunto representativo de microservicios desplegados en Kubernetes**, al menos:
 - servicio de autenticación/usuarios (donde se emite el JWT),
 - frontend,
 - el gateway/BFF o API pública que consume el frontend,
 - un servicio de negocio que dependa de la identidad (por ejemplo, carrito/checkout u órdenes),
 - los servicios que representen integraciones externas y FX, si el equipo decide incluirlos ya en el clúster.
3. **Depende de configuración externa** (Config Maps y Secrets) en lugar de valores “quemados” en el código o en los YAML de despliegue.
4. **Se puede desplegar y actualizar desde el pipeline**, no solo con comandos manuales locales.
5. **Permite verificar el uso de JWT** desde el punto de vista de un usuario final (frontend) y desde el punto de vista de los servicios (validación del token para acceder a recursos).

3.2 Kubernetes: despliegue y operación

3.2.1 Manifiestos obligatorios

El repositorio debe contener un directorio claramente identificable (por ejemplo `k8s/` o `deploy/k8s/`) con los manifiestos necesarios para describir el despliegue. Como mínimo:

- Un **Deployment** por cada componente que se va a desplegar en el clúster (frontend, gateway/BFF, servicio de autenticación, servicio de negocio seleccionado, etc.).
- Un **Service** por cada Deployment, de forma que cada microservicio tenga un nombre DNS estable dentro del clúster.
- Uno o varios recursos **Ingress** para exponer hacia Internet el frontend y el punto de entrada de la API (gateway/BFF o servicio público), con rutas claras y, si se aplica, prefijos de versión.
- Uno o varios **ConfigMaps** para agrupar configuración no sensible:
 - direcciones de otros servicios dentro del clúster,
 - URLs de APIs externas,
 - timeouts por defecto,
 - flags de características, etc.
- Uno o varios **Secrets** para agrupar información sensible:
 - secretos usados para firmar/verificar JWT,
 - credenciales de base de datos,
 - credenciales del registry de contenedores,
 - cualquier otro valor que no deba aparecer en texto plano.

Los manifiestos deben ser **completos y consistentes**: un Deployment que referencia un ConfigMap o Secret debe indicar claramente los `env` o `envFrom` correspondientes; los Servicios deben apuntar a los labels correctos; los Ingress deben apuntar a Servicios existentes, etc.

3.2.2 Probes, recursos y rollout/rollback

Cada Deployment que se utilice para evaluación debe incluir configuraciones mínimas de operación:

- **Readiness probe:**
 - Debe permitir saber cuándo un pod está listo para recibir tráfico.
 - Puede ser HTTP (por ejemplo, `/health/ready`), TCP o de comando, según el servicio.
 - Debe estar documentada su ruta y comportamiento.
- **Liveness probe:**
 - Debe permitir que Kubernetes detecte si el contenedor se quedó colgado o en un estado no recuperable.
 - Al igual que la readiness, debe ser coherente con el servicio y su documentación.
- **Requests y limits de CPU y memoria:**
 - Deben estar configurados al menos en los servicios más críticos (gateway/BFF, autenticación y el servicio de negocio que se use para pruebas).
 - No se busca una configuración perfecta, sino que los recursos estén explícitos y razonados.
- **Rollout y rollback:**

Debe existir una forma documentada de actualizar una versión y, si algo sale mal, volver a la versión anterior.

- Se espera que el equipo documentó al menos un flujo práctico, por ejemplo:
 - comando de despliegue (aplicar nuevos manifests o cambiar una imagen),
 - comando para ver el estado del rollout,
 - comando para revertir si se detecta un problema.

3.3 Seguridad con JWT

3.3.1 Servicio de autenticación

El servicio responsable de usuarios y autenticación debe:

- Exponer uno o más endpoints de autenticación (por ejemplo, `/auth/login`, `/auth/register`) que:
 - reciban credenciales y devuelvan un token JWT válido,
 - apliquen validaciones mínimas (credenciales obligatorias, usuario/bloqueo si aplica, etc.).
- Construir tokens JWT que incluyan al menos:
 - un identificador de usuario,
 - información de roles o permisos, cuando estos existan,
 - información de expiración (`exp`).
- Implementar la validación de tokens emitidos:
 - verificación de firma usando un secreto o clave configurada en un Secret,
 - verificación de que el token no está expirado,
 - verificación de que los claims mínimos existan.

La configuración de llaves/secrets de JWT debe **provenir de Secrets en Kubernetes** y de variables de entorno en entornos locales, nunca estar hardcodeada.

3.3.2 Autorización en microservicios y frontend

Los microservicios que expongan endpoints que manipulen datos ligados a un usuario (por ejemplo, carrito, órdenes, pagos, notificaciones personales) deben:

- Requerir un JWT válido para operaciones que dependan de la identidad.
- Verificar el token en cada petición protegida, extrayendo el identificador de usuario y/o roles.
- Rechazar accesos no autorizados (por ejemplo, con `401 Unauthorized` o `403 Forbidden` según corresponda).

El frontend, por su parte, debe:

- Realizar el proceso de login y recibir el token.

- Almacenar el token de manera apropiada (para efectos del curso no se exige un mecanismo específico, pero debe explicarse la decisión).
- Incluir el token en las peticiones que se hagan a los endpoints protegidos (por ejemplo, en la cabecera `Authorization: Bearer <token>`).
- Reflejar correctamente los estados típicos:
 - usuario no autenticado,
 - usuario autenticado,
 - error de autenticación (credenciales inválidas, token expirado, etc.).

3.4 CI/CD: de la imagen al despliegue

En esta fase el pipeline deja de ser únicamente de **integración continua** y se extiende a un ciclo de **integración y despliegue**.

3.4.1 Pipeline mínimo obligatorio

El pipeline debe tener, al menos, las siguientes etapas claramente diferenciadas:

1. **Build**
 - Construir las imágenes Docker de los servicios relevantes (microservicios y, si aplica, frontend).
 - Seguir una convención de tags que permita identificar la versión y, cuando aplique, la rama de origen.
2. **Test**
 - Ejecutar pruebas automáticas (unitarias u otras que el equipo tenga definidas).
 - Detener el pipeline si alguna prueba falla.
3. **Verificación de contratos e infraestructura** (puede ser parte de la misma etapa o separada, ver 3.5).
4. **Publicación**
 - Publicar las imágenes construidas en un registry.
 - Usar credenciales inyectadas mediante variables de entorno o secretos del sistema de CI/CD (no deben aparecer en el repositorio).
5. **Despliegue a Kubernetes**
 - Ejecutar los comandos necesarios para actualizar el clúster (por ejemplo, `kubectl apply -f k8s/`).
 - Dejar evidencia en los logs del pipeline de qué se desplegó y con qué versión.

3.4.2 Publicación en registry

La publicación debe cubrir, al menos, las imágenes que se usan en el clúster. Se espera que:

- Los nombres de imagen sigan una convención coherente (por ejemplo, `<registry>/<grupo>/servicio:<versión>`).
- El pipeline utilice un usuario/clave o token configurado como secreto del sistema de CI/CD.

- El README o la documentación de operación expliquen:
 - qué registry se utiliza,
 - qué imágenes se generan,
 - cómo se relacionan esas imágenes con los manifiestos de Kubernetes (por ejemplo, mediante la referencia en el campo `image` del Deployment).

3.4.3 Despliegue automatizado a Kubernetes

La etapa de despliegue debe:

- Ser reproducible (un commit con el mismo pipeline debe producir el mismo efecto, salvo cambios externos).
- Aplicar los manifests del repositorio, no manifests distintos almacenados fuera del control de versión.
- Dejar en los logs:
 - el clúster/entorno al que se desplegó,
 - el resultado de la aplicación de manifests (éxito/errores).

3.5 Contratos y validación en CI

3.5.1 OpenAPI / JSON Schema

Cada servicio que exponga APIs REST hacia el frontend o hacia otros equipos debe tener su contrato OpenAPI actualizado y almacenado en el repositorio. Para payloads complejos, se recomienda utilizar también JSON Schema.

En esta fase se espera que:

- La ubicación de los contratos sea clara (por ejemplo, un directorio `contracts/` con subdirectorios por servicio).
- La versión de los contratos sea explícita (por ejemplo, `v1`, `v2` en rutas o en el `info.version` del OpenAPI).
- Los cambios en los contratos se documentan con algún mecanismo de decisión (por ejemplo, ADRs en `/docs`).

3.5.2 Validaciones automáticas en pipeline

El pipeline debe incluir una validación automática de contratos que, como mínimo:

- Verifique que los archivos OpenAPI son sintácticamente válidos (sin errores de formato).
- Opcionalmente, puede detectar cambios incompatibles si el equipo incorpora herramientas para ello.
- Falle la pipeline si se detectan contratos inválidos o mal formados.

En la documentación debe indicarse qué herramientas se usaron para esta validación (linter, validador, etc.) y cómo ejecutar el mismo proceso en local.

3.6 Documentación de arquitectura y operación

Además del código y los pipelines, se requiere una documentación que explique:

- **Vista de despliegue en Kubernetes:**
 - diagrama de servicios y relaciones,
 - qué despliegues existen,
 - qué Ingress exponen el sistema,
 - cómo se conecta el frontend con la API.
- **Estrategia de seguridad JWT:**
 - qué servicio emite los tokens,
 - qué servicios los validan,
 - qué endpoints están protegidos y cuáles son públicos,
 - cómo lo refleja el frontend.
- **Estrategia de entrega:**
 - cómo se conecta el sistema de CI/CD con el registry y el clúster,
 - qué ramas o tags disparan despliegues,
 - cómo se hace rollback en caso de error.
- **Supuestos y decisiones clave:**
 - decisiones de configuración que afecten a rendimiento o seguridad,
 - justificación de los recursos configurados en Kubernetes,
 - decisiones sobre qué servicios se despliegan en el clúster en esta fase.

4. Entregables

Para esta fase, el equipo debe entregar:

1. **Repositorio actualizado** con:
 - manifiestos Kubernetes en un directorio claramente identificado,
 - pipeline CI/CD configurado,
 - scripts o configuraciones auxiliares que se utilicen para el despliegue.
2. **Clúster en funcionamiento** con:
 - frontend accesible desde fuera,
 - API accesible a través de Ingress,
 - puntos protegidos con JWT,
 - al menos un flujo de negocio completo que pueda probarse (por ejemplo, login → agregar al carrito → checkout → consulta de órdenes).
3. **Documentación en /docs** que cubra:
 - vista de arquitectura de despliegue,
 - descripción del flujo JWT,
 - descripción del pipeline y pasos de despliegue,
 - guía rápida para probar la fase (qué comandos ejecutar y qué endpoints visitar).

5. Restricciones técnicas

En esta fase:

- La orquestación principal a evaluar es **Kubernetes**; Docker Compose se mantiene como entorno de desarrollo local, pero no es el foco de evaluación.
- La configuración sensible (claves JWT, credenciales de BD, credenciales de registry) debe movilizarse a **Secrets** y no puede quedar en texto plano en el repositorio.
- Los despliegues deben hacerse a un clúster accesible para revisión (no es necesario que sea un entorno de producción, pero sí que sea verificable).

Las demás restricciones técnicas ya conocidas (lenguajes, base de datos, uso de Redis, etc.) se asumen vigentes y no se redefinen aquí.

Se considerará que la fase está **correctamente completada** cuando:

- Se puede levantar el sistema en Kubernetes siguiendo los pasos indicados, sin tener que “adivinar” configuraciones adicionales.
- El frontend cargue y se comunique con la API expuesta por el clúster.
- Existe un flujo demostrable de login con JWT y consumo de un endpoint protegido.
- El pipeline CI/CD termina en verde y logra desplegar una versión en el clúster sin intervención manual adicional distinta a la configuración inicial.
- La documentación es suficiente para que una persona distinta al equipo pueda seguirla y repetir el despliegue.

6. Documentación

La documentación mínima a actualizar o agregar en esta fase incluye:

- Guía de despliegue en Kubernetes (paso a paso).
- Descripción del pipeline CI/CD y de las variables/secrets que se necesitan.
- Diagrama de arquitectura de despliegue.
- Descripción del flujo JWT (con ejemplos de peticiones si es posible).
- Anexos con comandos de verificación:
 - ejemplos de llamadas `kubectl get/describe`,
 - ejemplos de endpoints de health,
 - ejemplos de peticiones autenticadas desde frontend o desde una herramienta de pruebas.

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Cálculo de nota individual

La nota individual se obtiene a partir de la nota grupal y un índice de aporte calculado con Git Fame. Para efectos de este curso, **los equipos son siempre de 5**, por lo que el “aporte esperado” por integrante es **20** en una escala de 0 a 100.

Valores (0 a 100) tomados de Git Fame

- **LOC**: aporte por líneas de código
- **COMMITTS**: aporte por commits
- **FILES**: aporte por archivos modificados

Cálculos

- $\text{ÍNDICE} = 0.5 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.3 * \text{FILES}$
- $\text{FACTOR} = \text{MIN}(\text{ÍNDICE} / 20, 1)$
- $\text{NOTA INDIVIDUAL} = \text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

Campo	Fórmula
ÍNDICE	$0.5 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.3 * \text{FILES}$
FACTOR	$\text{MIN}(\text{ÍNDICE} / 20, 1)$
NOTA INDIVIDUAL	$\text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

7.3 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** viernes 19 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.